

S.Y.B.Sc.(Data Science)

Sem- IV

Subject: Data Structure-II

Unit1 : Tree

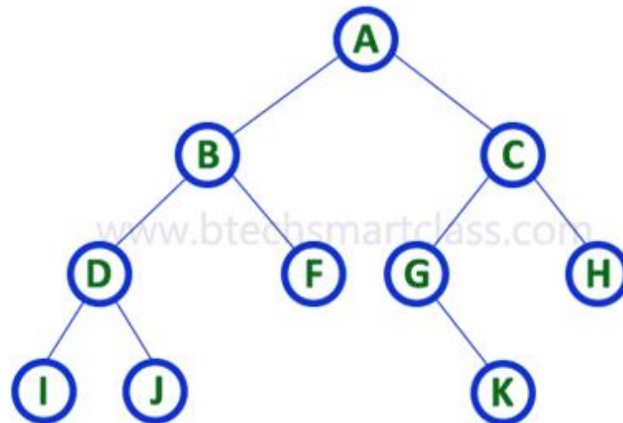
By,

Mrs. Reshma Masurekar

***Dr. D. Y. Patil Arts, Commerce and Science College,
Pimpri, Pune 18***

Representation of Tree

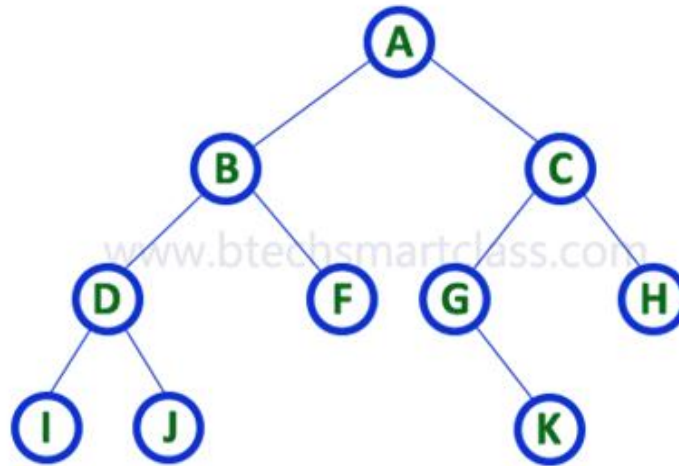
- A tree data structure is represented in following 2 ways:
 - Using an Array Representation(Sequential/Linear/Static)
 - Using a Linked List Representation (Link/Dynamic)
- Linked representation is more popular than Sequential representation due to
 - Natural implementation in linked storage
 - More convenient for insertion and deletion



Using an Array Representation

- In array representation of a binary tree, **one-dimensional array (1-D Array)** is used to represent a binary tree.
- Static representation of tree needs sequential numbering of the nodes starting with nodes on level zero, then those on level one and so on.
- A block of memory for an array is to be allocated.
- Nodes of the tree are stored level by level, starting from the zero level where the root node is present.
- The root node is stored in the first memory location as the first element in the array.
- To represent a binary tree of depth 'n' using array representation, we need one dimensional array with a maximum size of $2^{n+1} - 1$.

Using an Array Representation

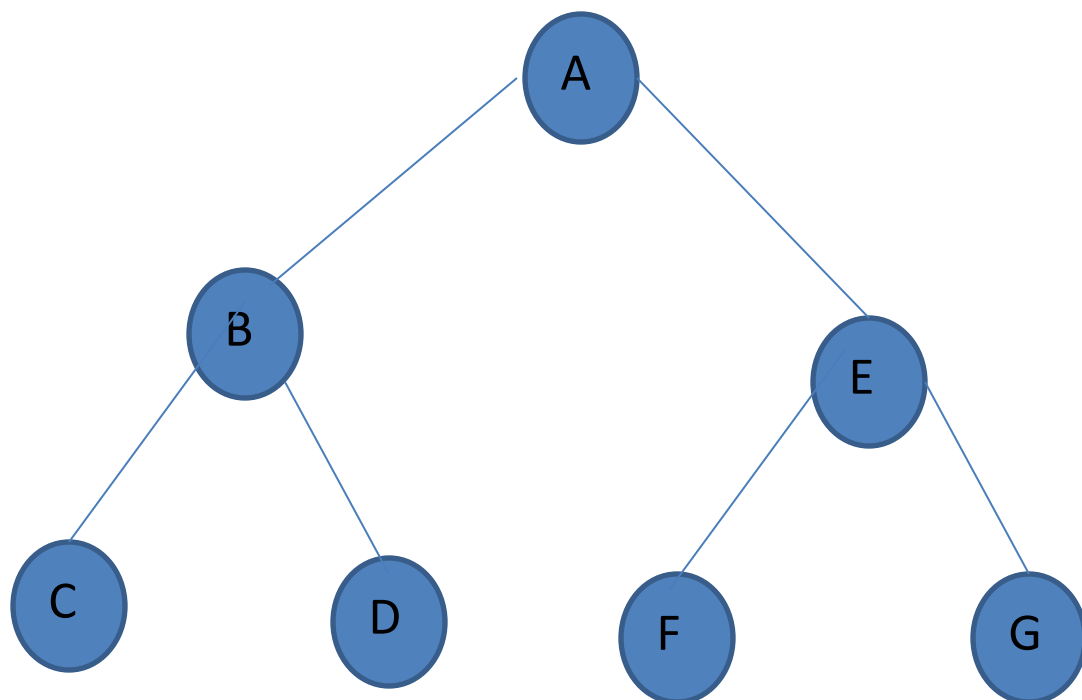


- Consider the above example of a binary tree and it is represented as follows...

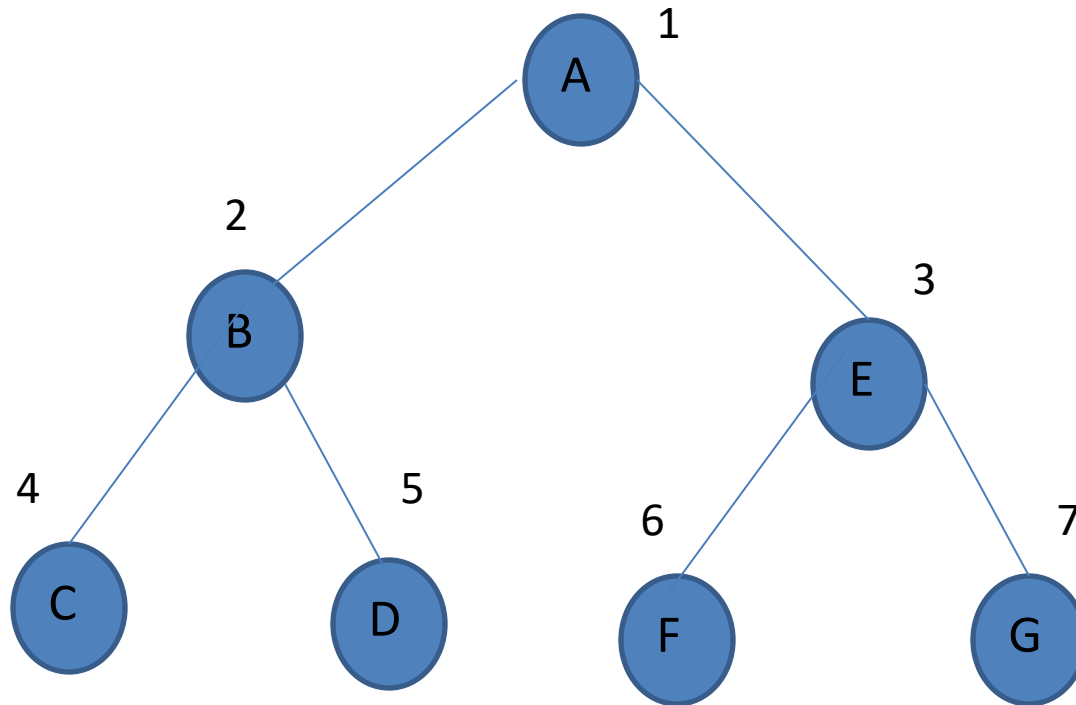


- To represent a binary tree of depth 'n' using array representation, we need one dimensional array with a maximum size of $2^{n+1} - 1$.

- Following rules are used to find the location of any “i”th node of a tree-
- For any node with index i , where i is , $1 \leq i \leq n$
 - a. $\text{Parent}(i) = (i/2)$ if $i \neq 1$
If $i=1$ then it is root which has no Parent.
 - b. $\text{Lchild}(i) = 2*i$, if $2i \leq n$
If $2*i > n$ then i has no left child.
 - c. $\text{Rchild}(i) = 2*i + 1$ if $2i+1 \leq n$
If $(2*i+1) > n$ then i has no Right child.



1	2	3	4	5	6	7	8
A	B	E	C	D	F	G	



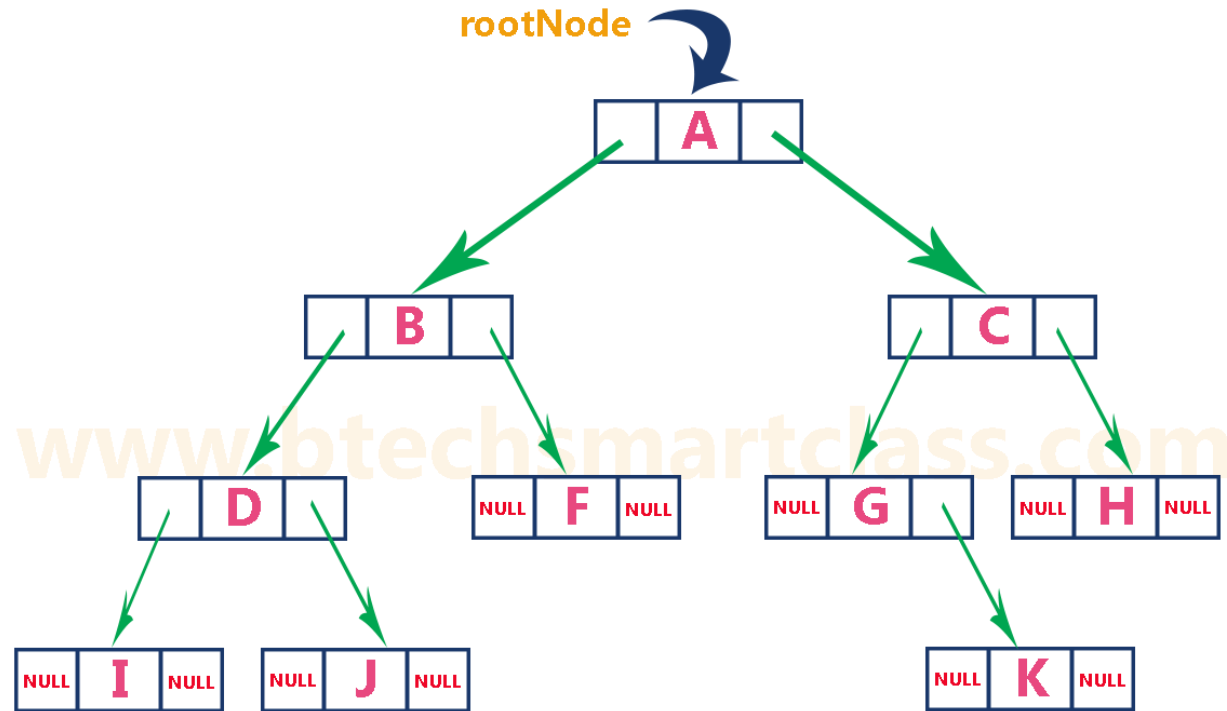
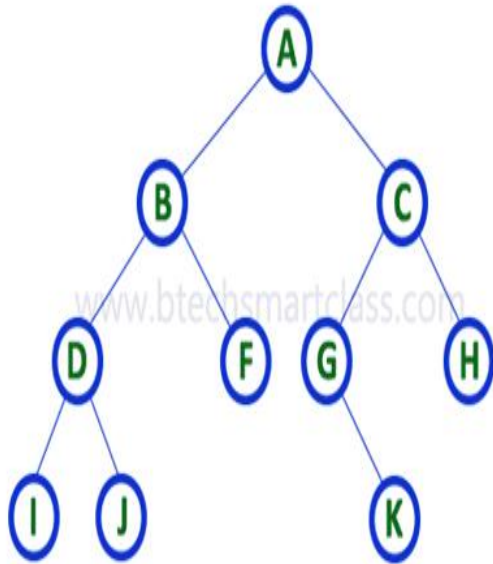
Parent(i) = (i/2)	Parent(D)=(5/2) = 2 i.e. B
LChild(i) = 2i	LChild(B)=2*2 = 4<7 i.e C
RChild(i) = 2i+1	RChild(B)=2*2+1 = 5<7 i.e. D
LChild(i) = 2i	Lchild(C)=4*2 = 8<7 = false => No Left Child Exist
RChild(i) = 2i+1	Rchild(C)=2*4+1 = 9<7 = false => No Right Child Exist

Using a Linked List Representation

- A doubly linked list is used to represent a binary tree. In a double linked list, every node consists of three fields. First field for storing left child address, second for storing actual data and third for storing right child address. In this linked list representation, a node has the following structure...



Binary tree represented using Linked list representation



Using a Linked List Representation

Insert data into BST

1. Compare the value with the current node.
2. If the node is None, create a new node with the value and return it.
3. If the value is equal to the node's data, indicate that the value already exists and return.
4. If the value is less than the node's data, recursively insert it into the left subtree.
5. If the value is greater than the node's data, recursively insert it into the right subtree.

Deleting Node from BST

- **Case 1: Node has No Children (Leaf Node)**

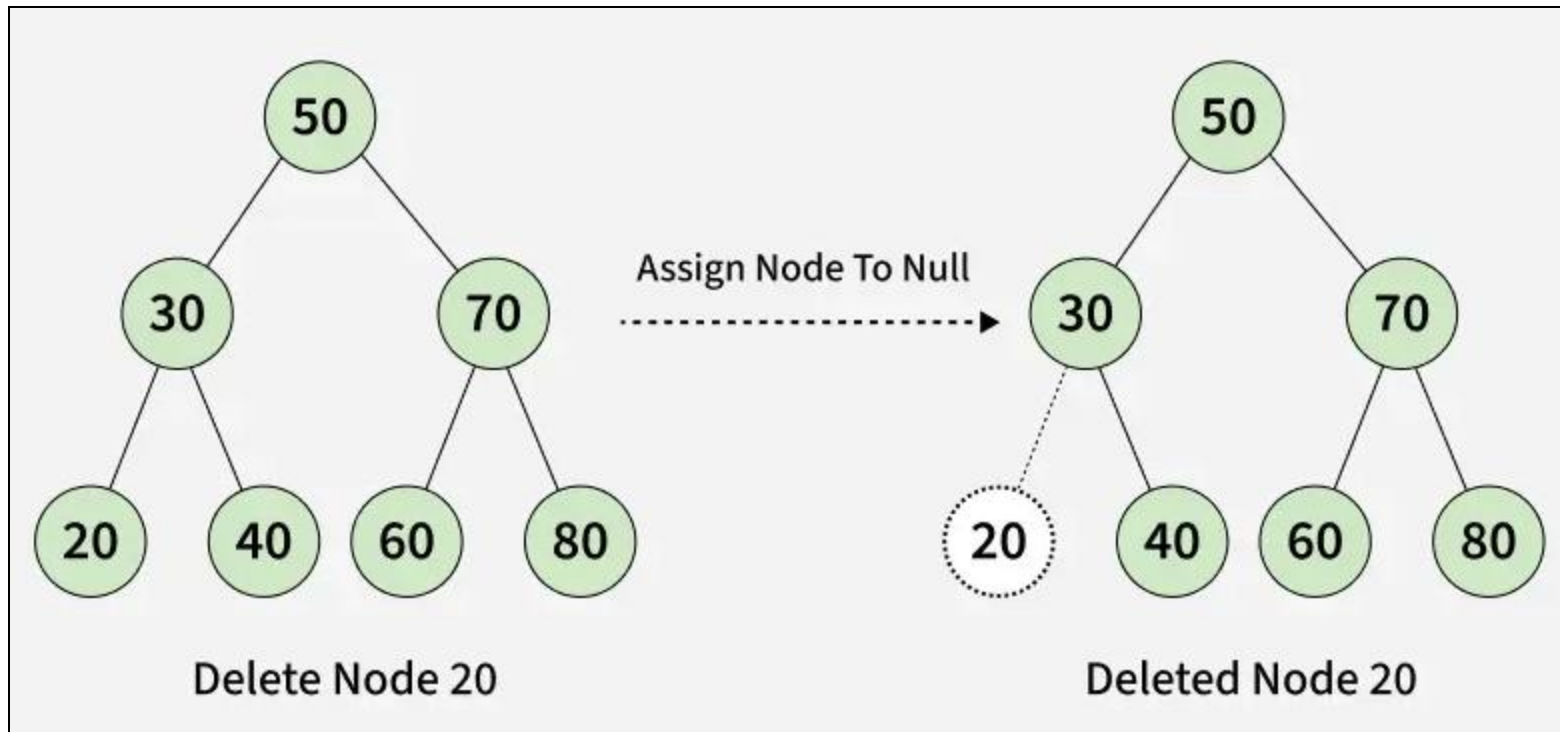
If the target node is a leaf node, it can be directly removed from the tree since it has no child to maintain.

- **Case 2: Node has One Child(Left or Right Child)**

If the target node has only one child, we remove the node and connect its parent directly to its only child. This way, the tree remains valid after deletion of target node.

Case 1: Deleting a Leaf Node

If the node to be deleted is a leaf node (i.e., it has no children), simply remove the node from the tree.

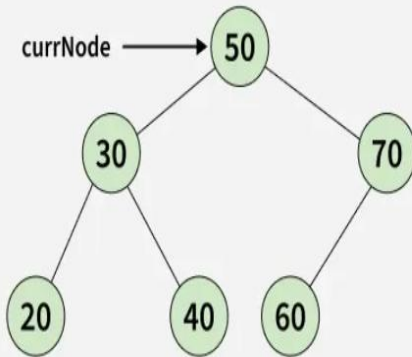


Case 2: Node has One Child(Left or Right Child)

If the target node has only one child, we remove the node and connect its parent directly to its only child. This way, the tree remains valid after deletion of target node.

01
Step

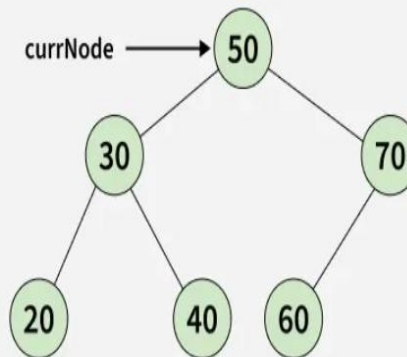
Consider the following BST



key = 70
(the node to be deleted)

02
Step

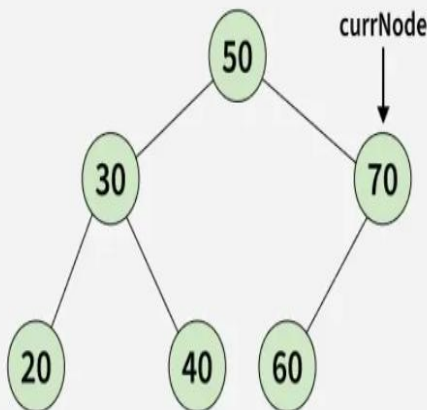
Comparing key with root node



Since 50 is smaller than key (70), move the pointer to the right child.

03
Step

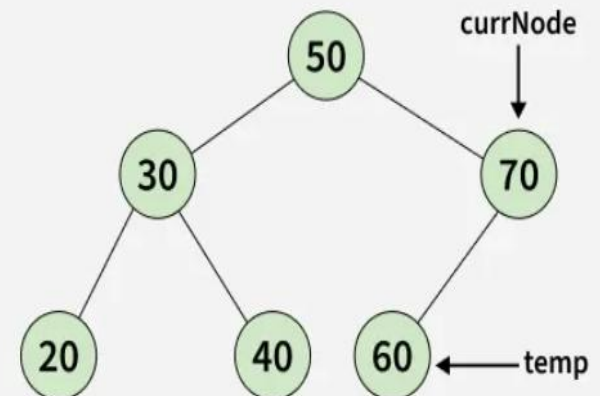
Comparing key with right child root node



Found the key node(70)

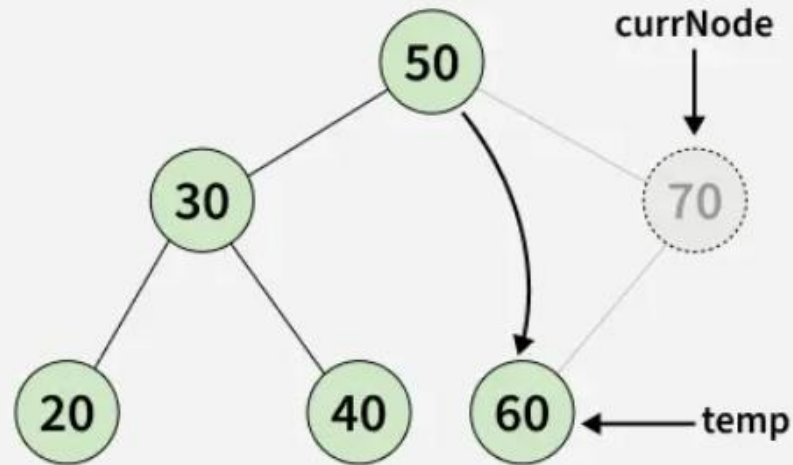
04
Step

Node 70 have only one child (temp=currNode->left).



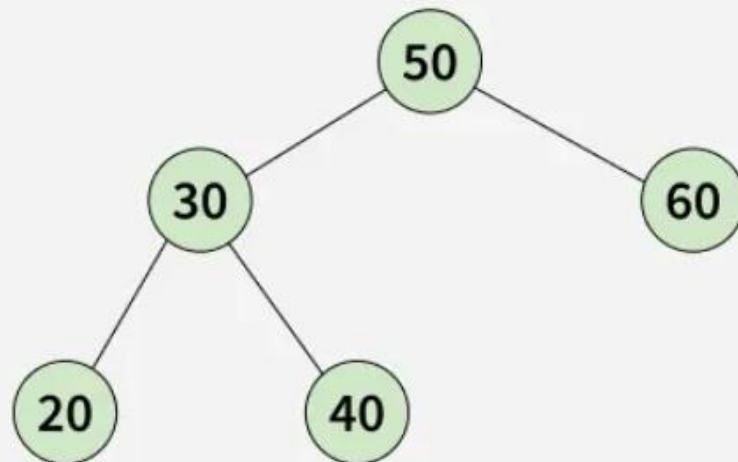
05
Step

Delete the node and return temp.



06
Step

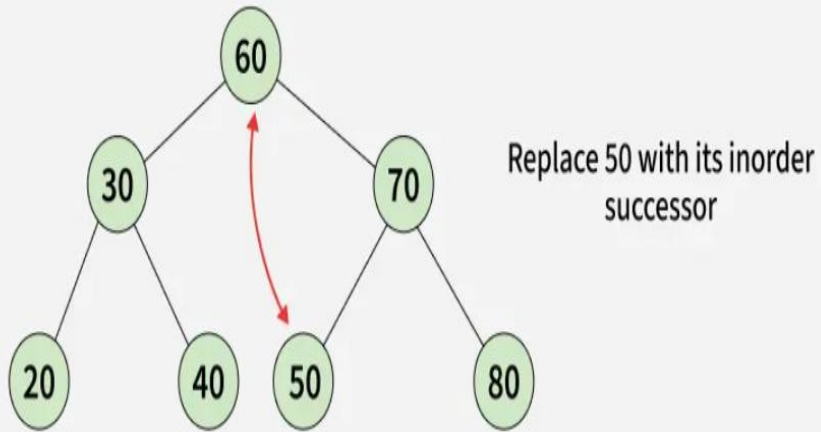
Final BST



- **Case 3: Node has Two Children**
 - If the target node has two children, deletion is slightly more complex.
 - To maintain the BST property, we need to find a replacement node for the target. The replacement can be either:
 - **The inorder successor** — the smallest value in the right subtree, which is the next greater value than the target node.
 - **The inorder predecessor** — the largest value in the left subtree, which is the next smaller value than the target node.
 - Once the replacement node is chosen, we replace the target node's value with that node's value, and then delete the replacement node, which will now fall under Case 1 (no children) or Case 2 (one child).
- **Note:** *Inorder predecessor can also be used*

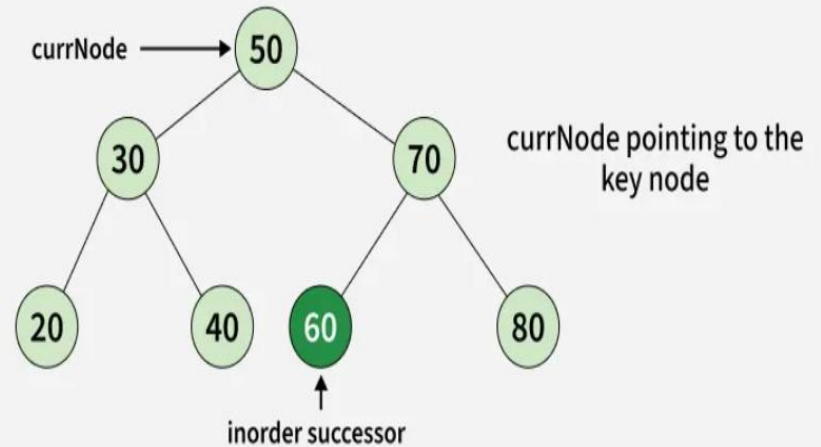
03
Step

Swapping the key node with its inorder successor



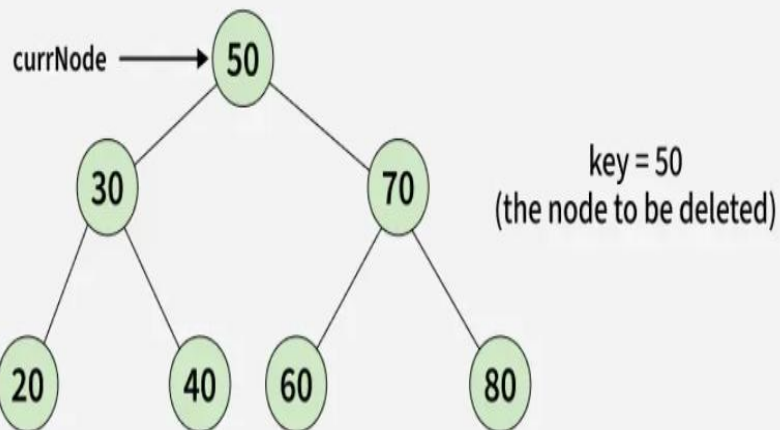
02
Step

Find the inorder successor



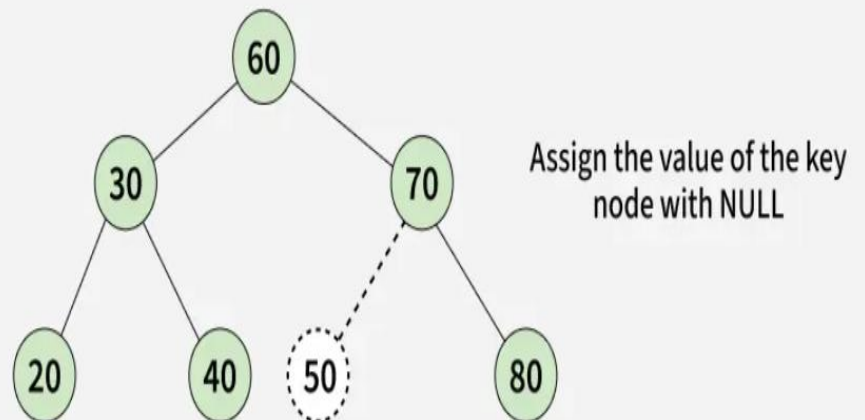
01
Step

Consider the following BST



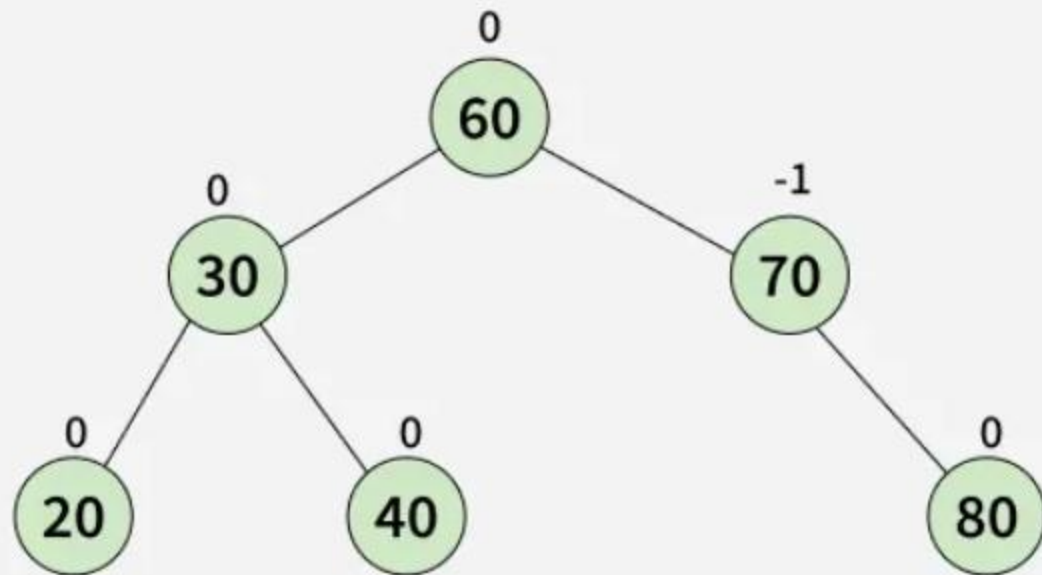
04
Step

Deletion of the key node (50)



05
Step

Final BST

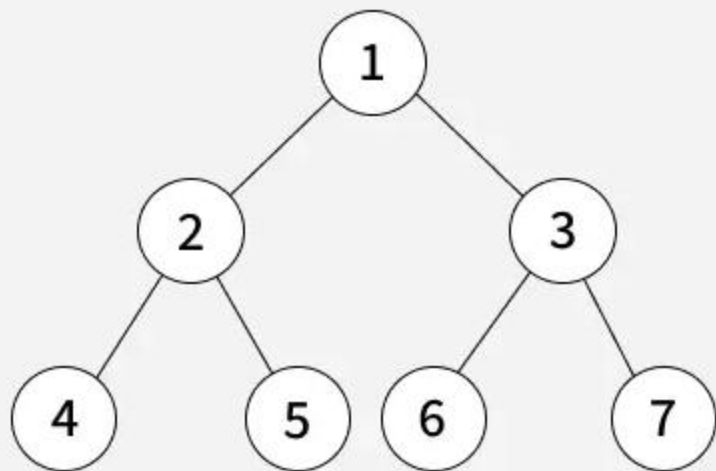


search

- To search for the number **key**, start at the root node. Then:
 - compare the Key value with the value of the root.
 - If it's equal, then found and return.
 - If it's smaller then search in the left subtree.
 - If it's greater then search in the right subtree.
 - Repeat the above step till no more traversal is possible
 - If at any iteration, key is found, return True. If the node is null, return False.

Types of Traversals

- Pre-order Traversal
- In-order Traversal
- Post-order Traversal



Inorder Traversal

4	2	5	1	6	3	7
---	---	---	---	---	---	---

Preorder Traversal

1	2	4	5	3	6	7
---	---	---	---	---	---	---

Postorder Traversal

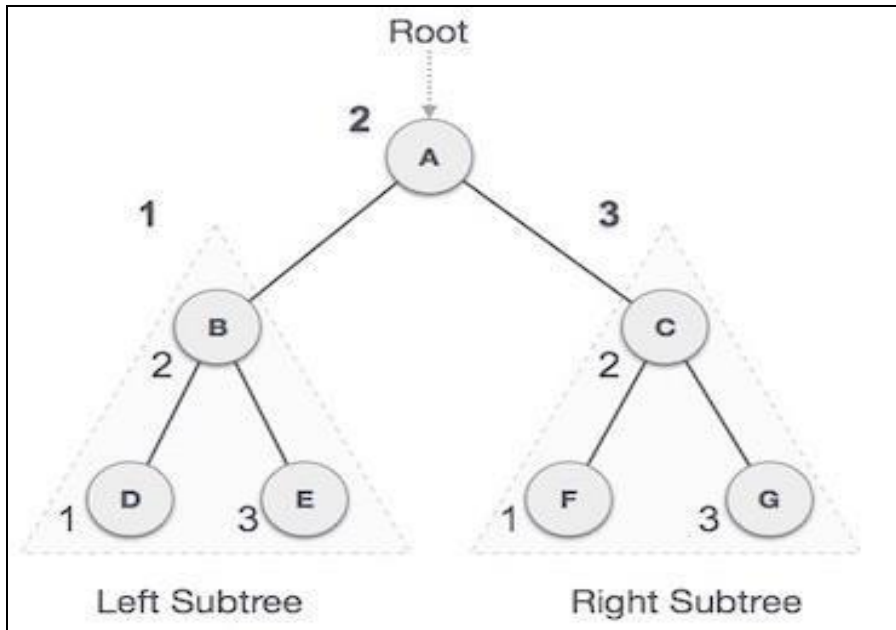
4	5	2	6	7	3	1
---	---	---	---	---	---	---

Level order Traversal

1	2	3	4	5	6	7
---	---	---	---	---	---	---

Inorder Traversal

- Inorder traversal visits the node in the order: **Left -> Root -> Right**
- In this traversal method, the left subtree is visited first, then the root and later the right sub-tree. We should always remember that every node may represent a subtree itself.
- If a binary tree is traversed **in-order**, the output will produce sorted key values in an ascending order.
- **Algorithm for Inorder Traversal:**
- Inorder(tree)
 - Traverse the left subtree, i.e., call Inorder(left->subtree)
 - Visit the root.
 - Traverse the right subtree, i.e., call Inorder(right->subtree)



Step 1 – Recursively traverse left subtree.

Step 2 – Visit root node.

Step 3 – Recursively traverse right subtree.

Inorder: D → B → E → A → F → C → G

function to perform inorder tree traversal

```
def Inorder(root):
```

```
    if root:
```

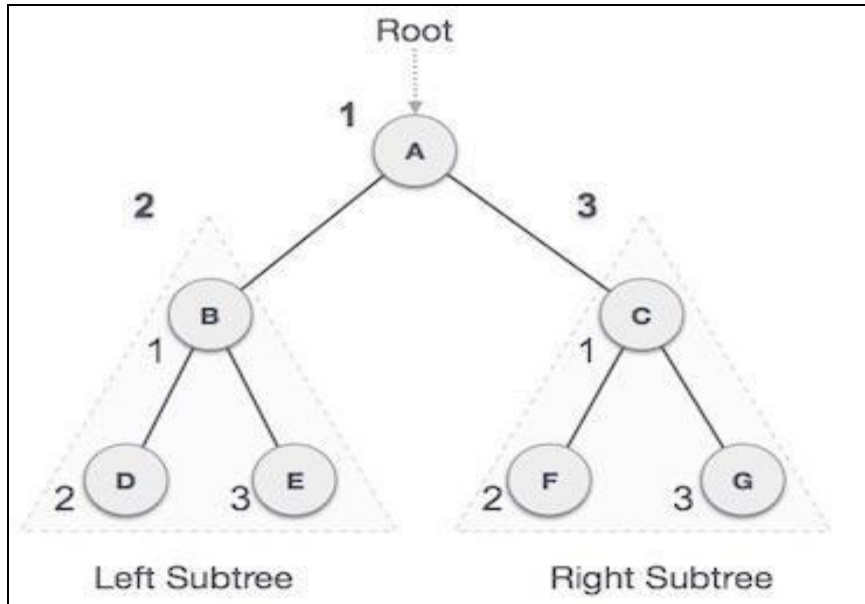
```
        Inorder(root.leftChild)
```

```
        print(root.data)
```

```
        Inorder (root.rightChild)
```

Preorder Traversal

- Preorder traversal visits the node in the order: **Root -> Left -> Right**
- In this traversal method, the root node is visited first, then the left subtree and finally the right subtree.
- **Algorithm for Preorder Traversal**
- Preorder(tree)
 - Visit the root.
 - Traverse the left subtree, i.e., call Preorder(left->subtree)
 - Traverse the right subtree, i.e., call Preorder(right->subtree)



Step 1 – Visit root node.

Step 2 – Recursively traverse left subtree.

Step 3 – Recursively traverse right subtree.

Preorder: A → B → D → E → C → F → G

function to perform preorder tree traversal

```
def Preorder(root):
```

```
    if root:
```

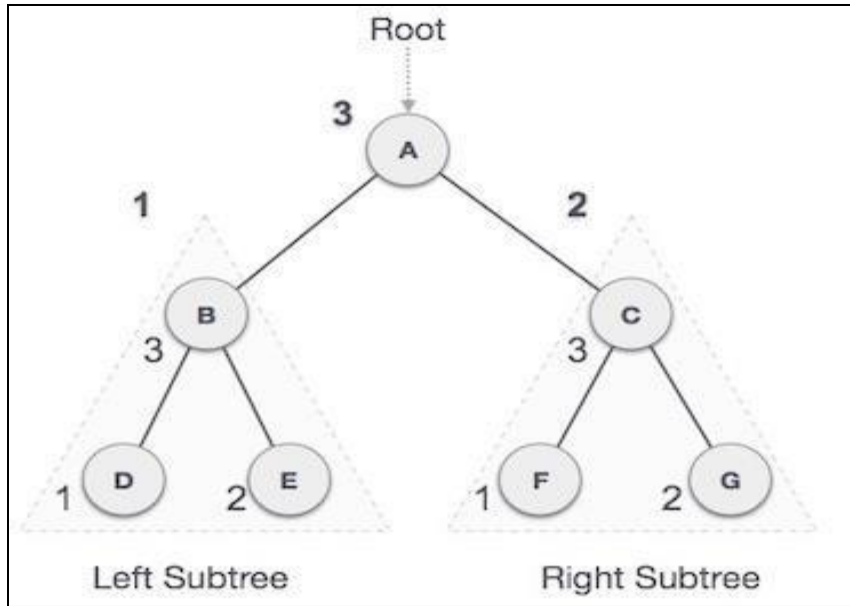
```
        print(root.data)
```

```
        Preorder(root.leftChild)
```

```
        Preorder (root.rightChild)
```


Postorder Traversal

- Postorder traversal visits the node in the order: **Left -> Right -> Root**
- In this traversal method, the root node is visited last, hence the name. First we traverse the left subtree, then the right subtree and finally the root node.
- **Algorithm for Postorder Traversal:**
Postorder(tree)
 - Traverse the left subtree, i.e., call Postorder(left->subtree)
 - Traverse the right subtree, i.e., call Postorder(right->subtree)
 - Visit the root



Step 1 – Recursively traverse left subtree.
Step 2 – Recursively traverse right subtree.
Step 3 – Visit root node.

Postorder: D → E → B → F → G → C → A

function to perform preorder tree traversal

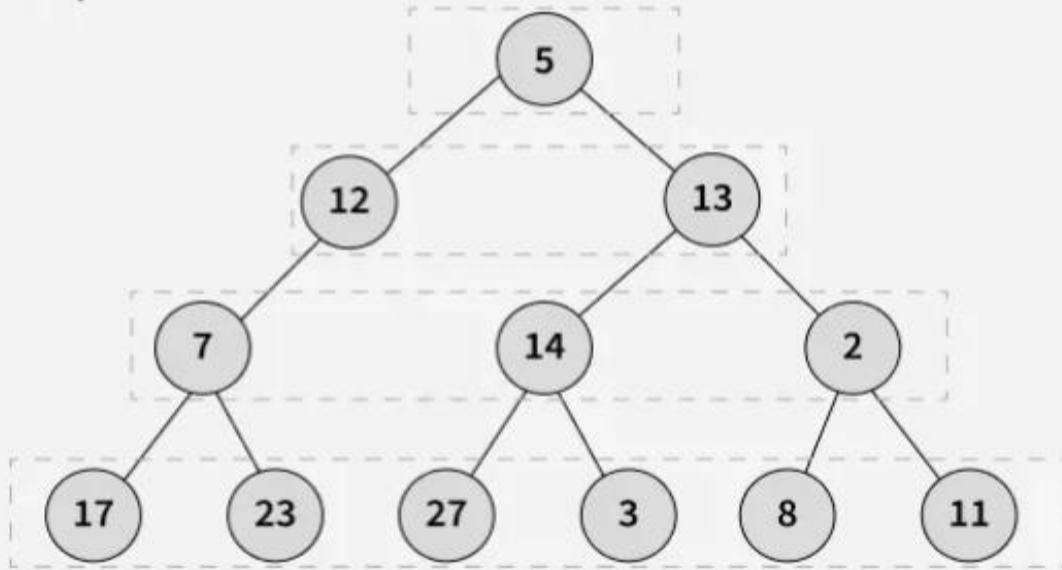
```
def Postorder(root):  
    if root:  
        Postorder(root.leftChild)  
        Postorder (root.rightChild)  
        print(root.data)
```

Level Order Traversal

- Level Order Traversal visits all nodes present in the same level completely before visiting the next level.
- **Algorithm for Level Order Traversal**
- LevelOrder(tree)
 - Create an empty queue Q
 - Enqueue the root node of the tree to Q
 - Loop while Q is not empty
 - Dequeue a node from Q and visit it
 - Enqueue the left child of the dequeued node if it exists
 - Enqueue the right child of the dequeued node if it exists .

01
Step

Enqueue the root node to the Queue.

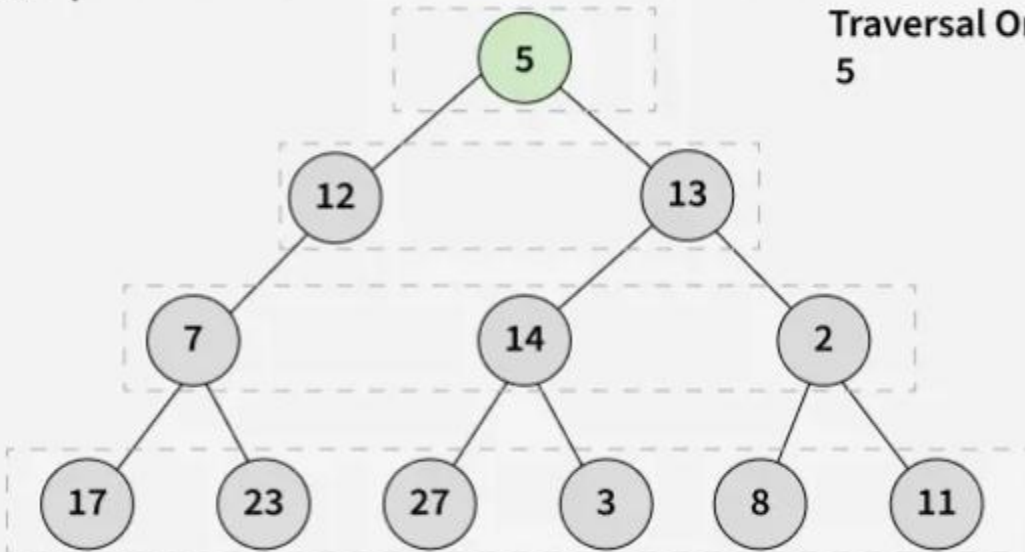


Queue

5		
---	--	--

02
Step

Dequeue the front element (**5**) from the Queue and Enqueue its left child (**12**) and right child (**13**) to the queue respectively.



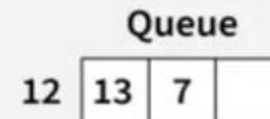
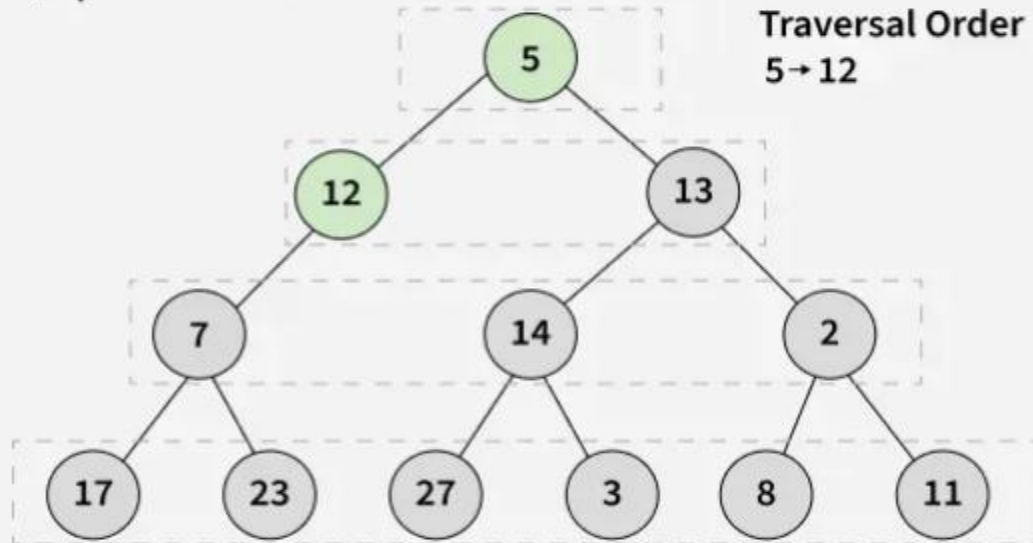
Traversal Order
5

Queue

5	12	13	
---	----	----	--

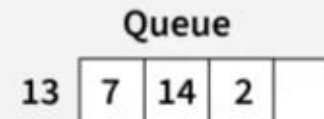
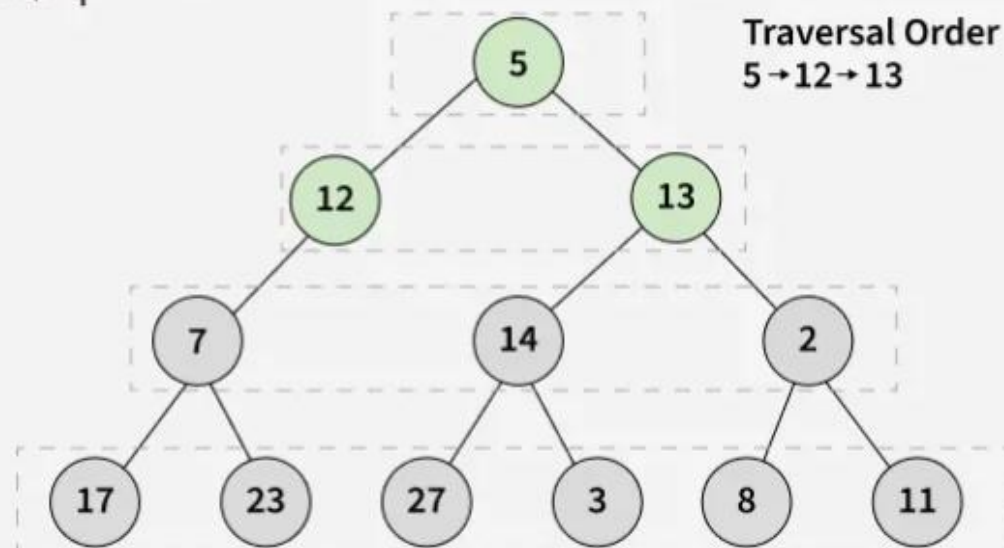
03
Step

Dequeue the front element (**12**) from the Queue and Enqueue its left child (**7**) to the queue.



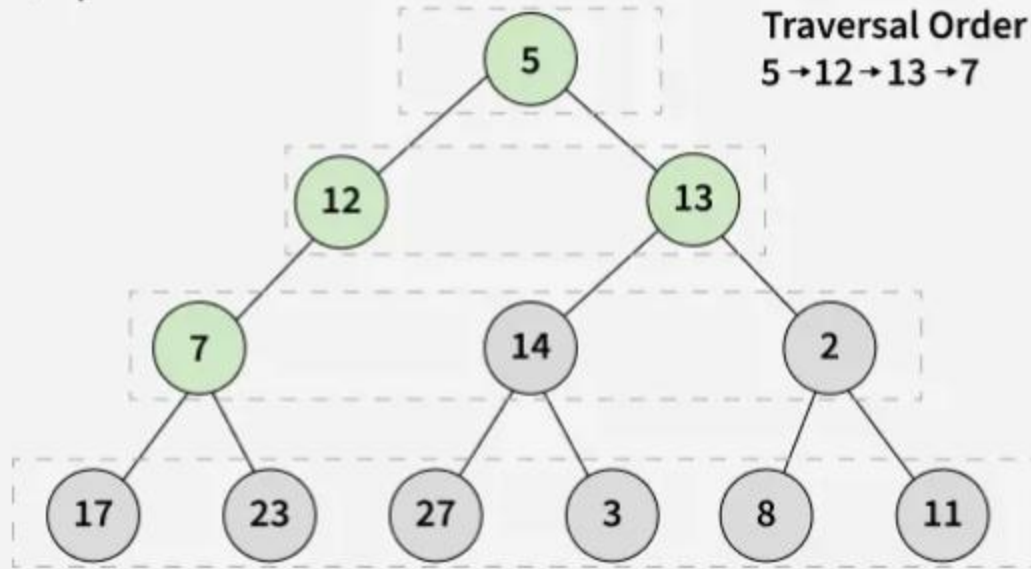
04
Step

Dequeue the front element (**13**) from the Queue and Enqueue its left child (**14**) and right child (**2**) to the queue respectively.



05
Step

Dequeue the front element (**7**) from the Queue and Enqueue its left child (**17**) and right child (**23**) to the queue respectively.

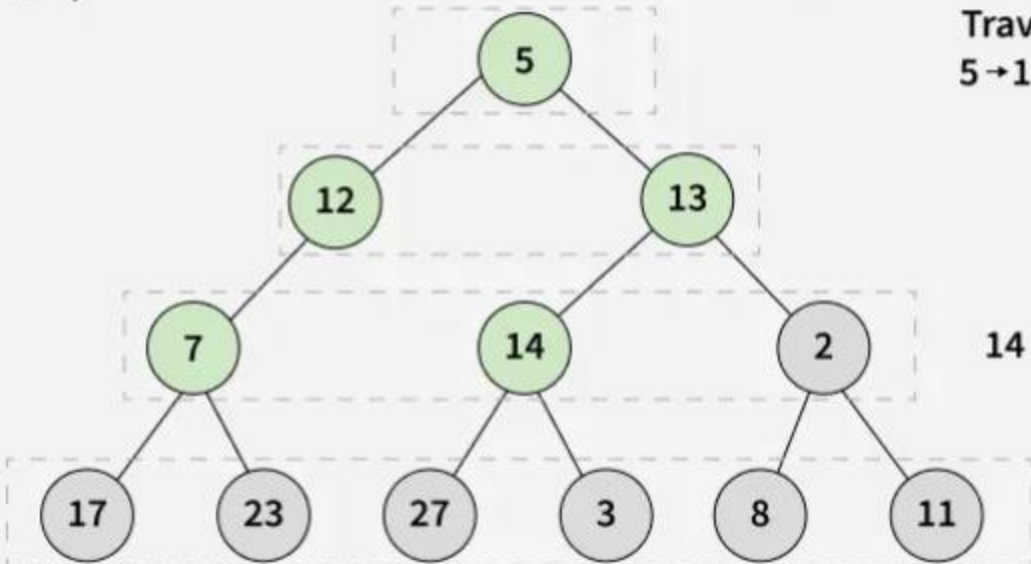


Queue

7	14	2	17	23	
---	----	---	----	----	--

06
Step

Dequeue the front element (**14**) from the Queue and Enqueue its left child (**27**) and right child (**3**) to the queue respectively.



Queue

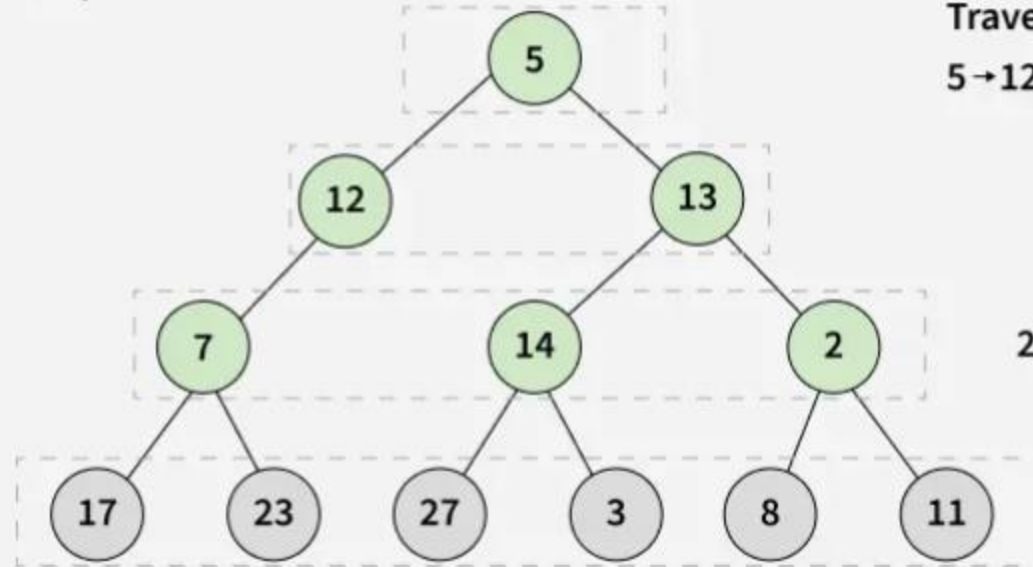
14	2	17	23	27	3	
----	---	----	----	----	---	--

07
step

Dequeue the front element (**2**) from the Queue and Enqueue its left child (**8**) and right child (**11**) to the queue respectively.

Traversal Order

5 → 12 → 13 → 7 → 14 → 2



Queue

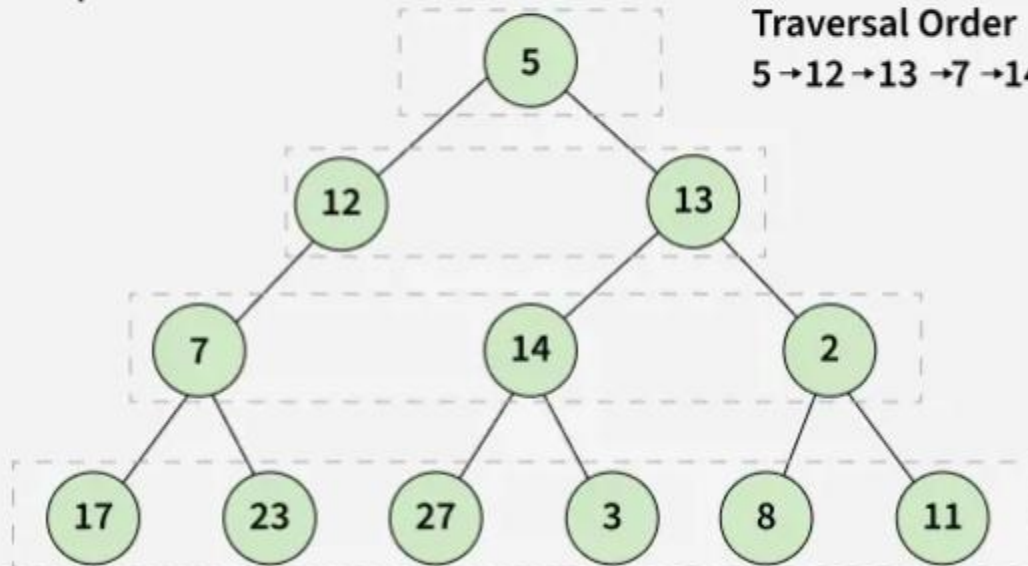
2	17	23	27	3	8	11	
---	----	----	----	---	---	----	--

08
step

Similarly Dequeue the rest of the nodes from the queue until the queue is empty.

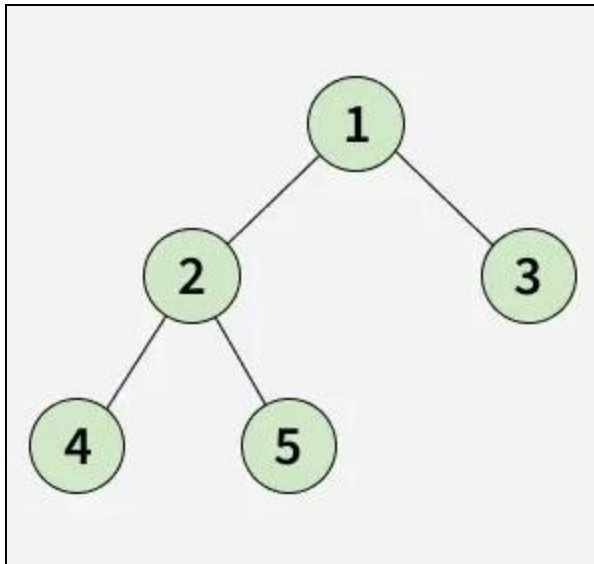
Traversal Order

5 → 12 → 13 → 7 → 14 → 2 → 17 → 23 → 27 → 3 → 8 → 11

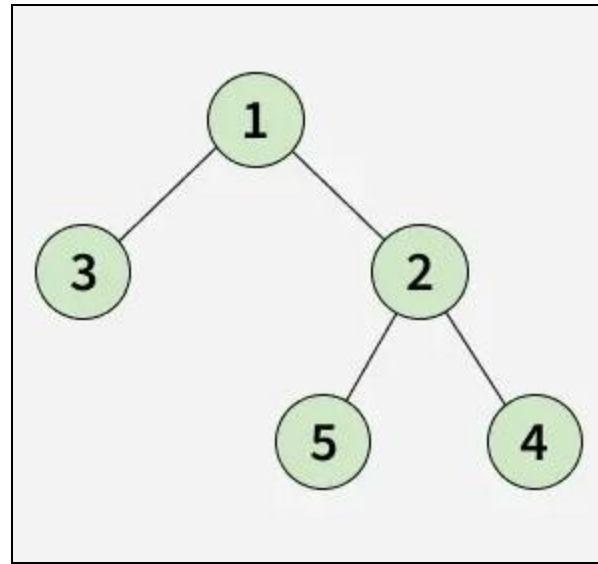


mirror() of BST

- A mirror of a binary tree is another tree in which the left and right children of every non-leaf node are interchanged.
- **Algorithm — Mirror (tree): Recursive**
 - Call Mirror for left-subtree i.e., Mirror(left-subtree)
 - Call Mirror for right-subtree i.e., Mirror(right-subtree)
 - Swap left and right subtrees.



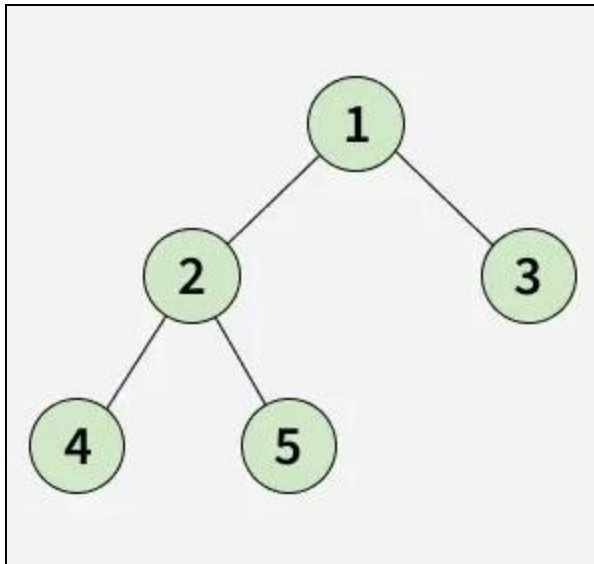
Original Tree



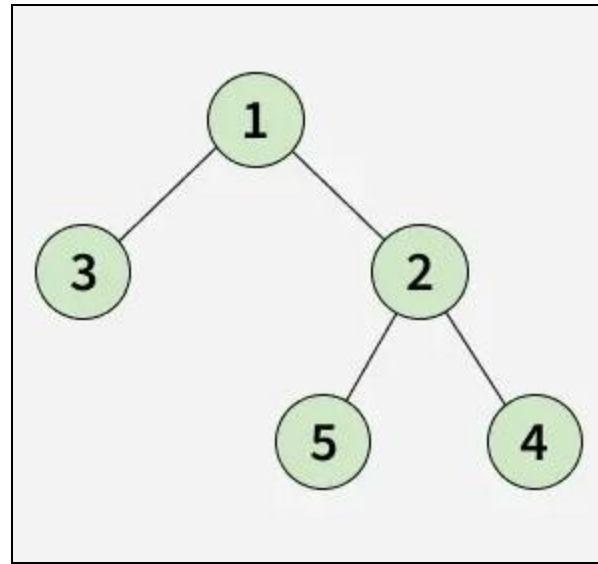
Mirror image of original tree

copy() of BST

- A mirror of a binary tree is another tree in which the left and right children of every non-leaf node are interchanged.
- **Algorithm — Mirror (tree): Recursive**
 - Call Mirror for left-subtree i.e., Mirror(left-subtree)
 - Call Mirror for right-subtree i.e., Mirror(right-subtree)
 - Swap left and right subtrees.



Original Tree



Mirror image of original tree

Time Complexity:

Operation	Best Case Complexity	Average Case Complexity	Worst Case Complexity
Search	$O(\log n)$	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(\log n)$	$O(n)$
Traversal (Inorder/ Preorder/ postorder)	$O(n)$	$O(n)$	$O(n)$

Where, n is the number of nodes in the tree.

Space Complexity: The space complexity for all the operations is $O(n)$